Operating System Fundamentals • Chapter 07

Memory Manager

How many programs share one RAM safely: address translation, swapping, contiguous allocation, paging and segmentation.

Chapter 7 of the notebook

BIG QUESTION

Ten programs, one RAM. How can each one believe it starts at address 0?

Today's route

What we'll cover today

1. **Logical vs physical addresses**
   And the MMU that translates them
2. **Swapping**
   Parking processes on disk
3. **Contiguous allocation**
   Holes, fits and fragmentation
4. **Paging**
   Fixed-size pages and frames
5. **Segmentation**
   Memory the way programmers see it
6. **Segmentation with paging**
   The MULTICS combination

Part 01

Logical vs physical addresses

THINK FIRST

"Seat 12 in row C" vs the seat's GPS coordinates. If the whole cinema moves, which one stays valid?

discuss for one minute before the next slide

Two kinds of address

What the CPU says vs what memory sees



Logical (virtual) address

Generated by the CPU. The program only ever sees these.



Physical address

The address the memory unit actually receives.

Compile-time or load-time binding

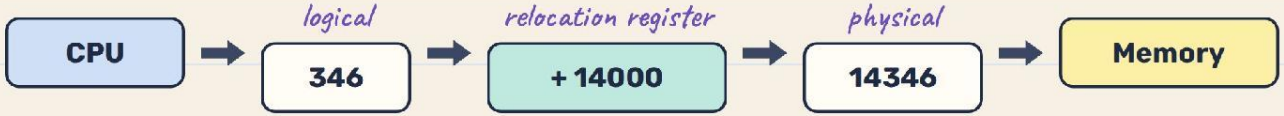
Logical = physical

Execution-time binding

Logical ≠ physical: translated on every access

Memory-Management Unit (MMU)

Hardware that translates every address



The rule

The value in the relocation (base) register is added to every address a user process sends to memory.

The illusion

The user program deals only with logical addresses. It never sees the real physical ones.

Pop quiz • click to reveal

The relocation register holds 20000. The CPU issues logical address 500. Which physical address reaches memory?

- A 500
- B 19500
- C 20500
- D 20000

Pop quiz • click to reveal

The relocation register holds 20000. The CPU issues logical address 500. Which physical address reaches memory?

- A 500
- B 19500
- C 20500
- D 20000

Answer C. Physical = relocation register + logical = 20000 + 500 = 20500.

Part 02

Swapping

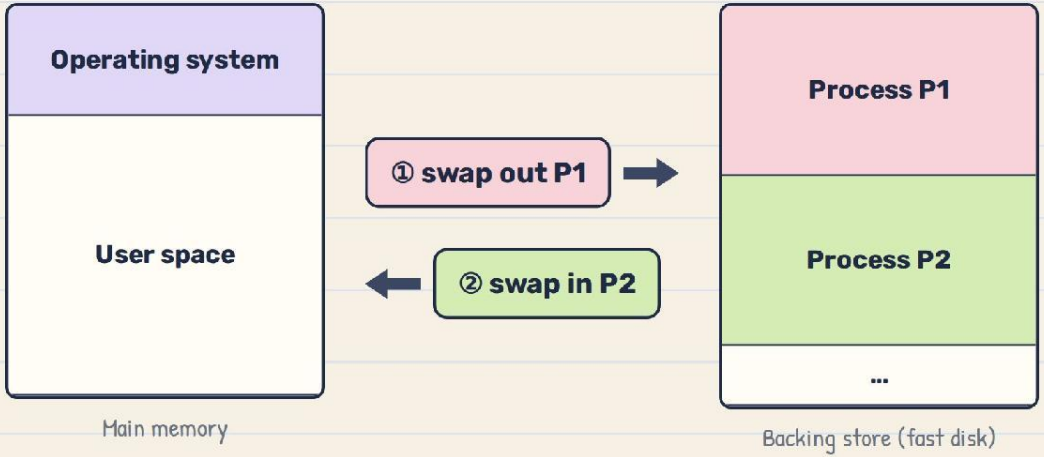
THINK FIRST

The restaurant is full. Could you ask a slow diner to wait at the bar so someone urgent can sit, then bring them back?

discuss for one minute before the next slide

Swapping

Park a process on disk, bring it back later



Roll out, roll in

Priority version: swap out a low-priority process so a higher-priority one can load and run.

Swapping

The fine print



Backing store

A fast disk with room for a copy of every process's memory.



Cost = transfer time

Most of the time goes into copying. Twice the memory takes twice as long to swap.



Still around

Modified versions of swapping are found in UNIX, Linux and Windows.

↺ Recall · time-sharing, Ch 1 · medium-term scheduler, Ch 4

Both only mentioned swapping. Here is how it works.

Part 03

Contiguous allocation

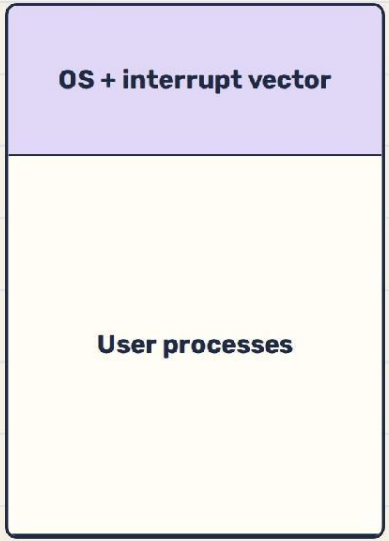
THINK FIRST

Parking cars of different sizes along one kerb. After a few leave, can a bus still find a space?

discuss for one minute before the next slide

Contiguous allocation

Each process gets one continuous block



Low memory at the top

Two partitions

The resident OS sits in low memory with the interrupt vector. User processes live in high memory.

Single-partition protection

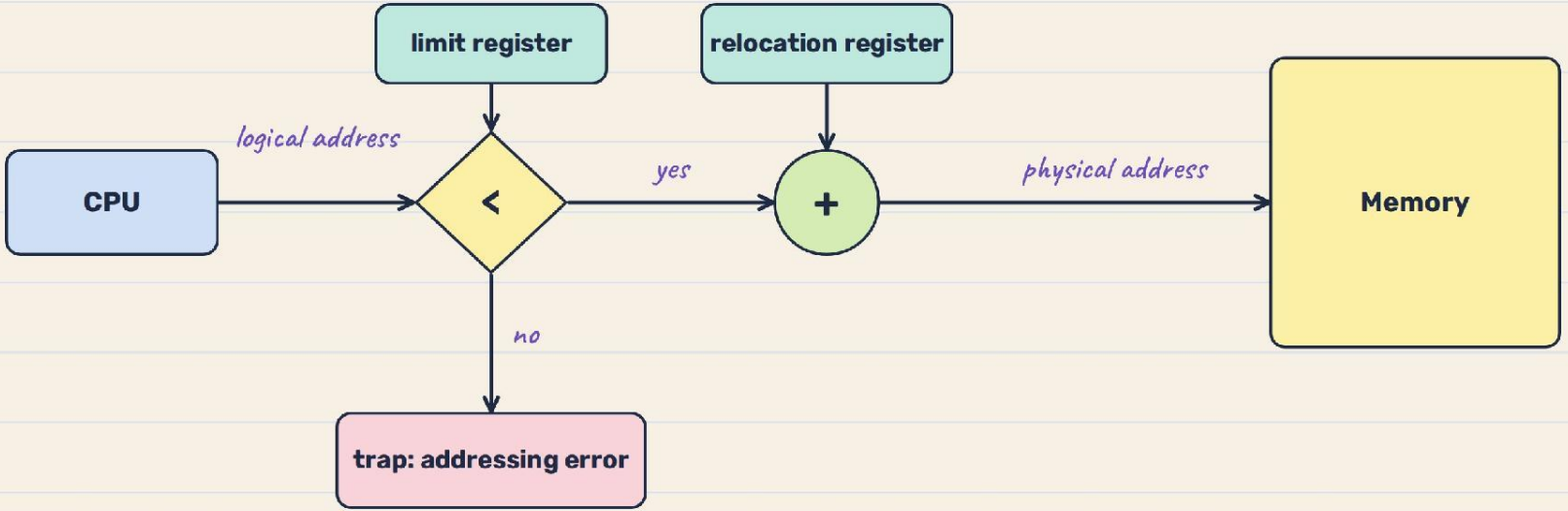
Relocation register = smallest physical address; limit register = range of logical addresses. Protects processes from each other and from the OS.

↺ Recall • base & limit concept, Ch 2

Ch 2 introduced the idea; next slide shows the exact hardware check.

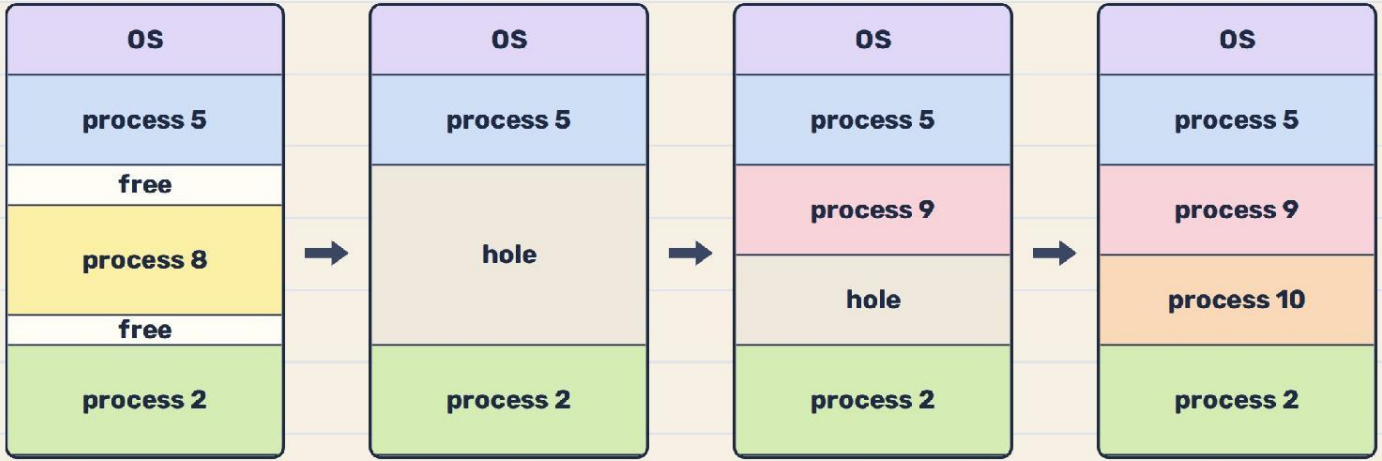
Hardware support

Check the limit, then add the base



Multiple partitions

Processes come and go, holes appear



The OS keeps two lists: **allocated partitions** and **free partitions (holes)**. A new process gets a hole big enough for it.

Dynamic storage allocation

Which hole should we pick?



First-fit

The first hole that is big enough. Fast.



Best-fit

The smallest hole that is big enough. Must search the whole list (unless sorted). Smallest leftover.



Worst-fit

The largest hole. Must also search the whole list. Largest leftover.



First-fit and best-fit beat worst-fit in both speed and storage utilisation.

Worked example

Holes: 100, 500, 200, 300, 600 KB

Request	First-fit	Best-fit	Worst-fit
212 KB	500 → 288 left	300 → 88 left	600 → 388 left
417 KB	600 → 183 left	500 → 83 left	500 → 83 left
112 KB	288 → 176 left	200 → 88 left	388 → 276 left
426 KB	must wait	600 → 174 left	must wait

here, only best-fit places all four requests

Fragmentation

Wasted memory, two flavours



External fragmentation

Enough total free memory exists, but it is not contiguous.

parking: 3 small gaps, no room for the bus



Internal fragmentation

The allocated block is slightly larger than requested; the extra inside the partition is unused.

a 5-seat taxi carrying 3 people



Compaction

Shuffle memory contents to put all free memory together in one large block. Reduces external fragmentation.

Pop quiz • click to reveal

There is 300 KB free in total, but split into three 100 KB holes, so a 250 KB process can't load. This is…

- A Internal fragmentation
- B External fragmentation
- C Swapping
- D A page fault

Pop quiz • click to reveal

There is 300 KB free in total, but split into three 100 KB holes, so a 250 KB process can't load. This is…

A Internal fragmentation

B External fragmentation

C Swapping

D A page fault

Answer B. Enough memory in total, just not contiguous: external fragmentation.

Part 04

Paging

THINK FIRST

A book isn't printed on one continuous scroll; it's pages. What if a program didn't need one continuous block of memory either?

discuss for one minute before the next slide

Paging

Cut memory into equal-sized pieces

Frames

Physical memory split into fixed-size blocks. Size is a power of 2, between 512 and 8192 bytes.

Pages

Logical memory split into blocks of the same size.

Page table

Translates each page to the frame it lives in.

To run n pages

Find n free frames anywhere and load the program. Keep track of all free frames.

no external fragmentation... but some internal (the last page is rarely full)

Address translation

Split the address: page number + offset



Page number p

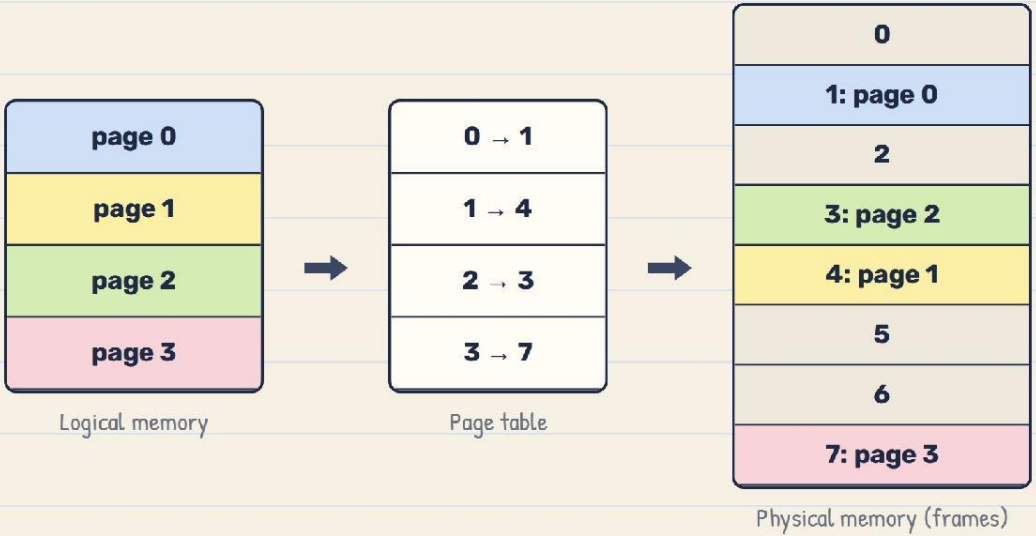
Index into the page table, which holds the base address (frame) of each page.

Page offset d

Combined with the frame's base address to form the physical address.

Paging example

Pages land in any free frame



Worked example

Page size = 4 bytes: translate by hand

Page	Frame
0	5
1	6
2	1
3	2

Logical	p = ÷4	d = mod 4	Physical = f×4 + d
0	0	0	5×4 + 0 = 20
3	0	3	5×4 + 3 = 23
4	1	0	6×4 + 0 = 24
13	3	1	2×4 + 1 = 9

try it yourself: where does logical address 6 go? (answer on the quiz)

Free frames

Before and after allocation

Before

- Free-frame list: 14, 13, 18, 20, 15
- A new process arrives with 4 pages

After

- Page 0 → frame 14
- Page 1 → frame 13
- Page 2 → frame 18
- Page 3 → frame 20
- Free-frame list: 15

Shared pages

One copy of the code, many users



The idea

Code that never changes (an editor, a compiler) is loaded once. Every process that uses it points to the same frames.



What stays private

Each process still gets its own data pages: your document is not my document.



Real life

One projector in a classroom shows the same slides to 40 students; each student keeps their own notebook.

Pop quiz • click to reveal

Page size 4 bytes, page table 0 → 5, 1 → 6, 2 → 1, 3 → 2. Where does logical address 6 go?

A 6

B 26

C 10

D 24

Pop quiz • click to reveal

Page size 4 bytes, page table 0 → 5, 1 → 6, 2 → 1, 3 → 2. Where does logical address 6 go?

A 6

B 26

C 10

D 24

Answer B. 6 ÷ 4 = page 1, offset 2. Page 1 → frame 6. Physical = 6 × 4 + 2 = 26.

Part 05

Segmentation

THINK FIRST

You think of your program as main, functions, a stack and arrays, not as "bytes 0 to 99,999". Could memory think that way too?

discuss for one minute before the next slide

Segmentation

Memory the way programmers see it



A program is a collection of segments

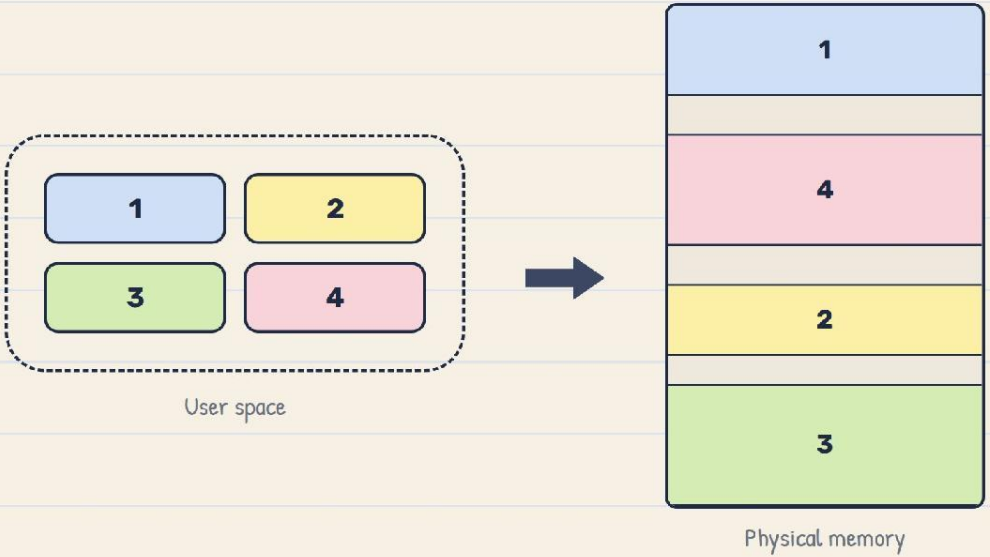
A segment is a logical unit, and each can have its own size.

main program | procedure | function | method | common block | stack

symbol table | arrays

Logical view

Segments scatter in physical memory



Segmentation architecture

Address = <segment, offset>

Segment table

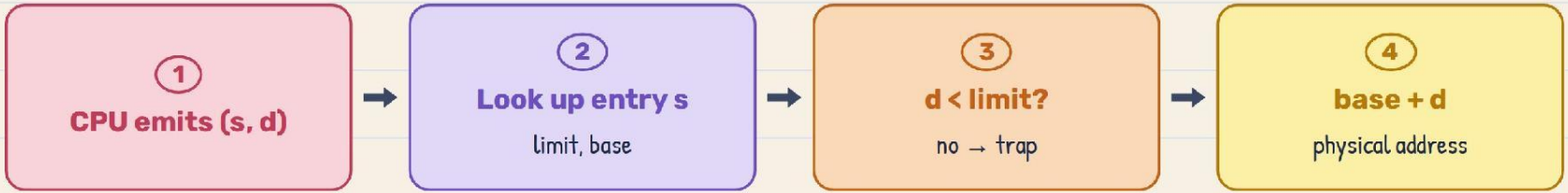
- **base:** starting physical address of the segment
- **limit:** length of the segment

Relocation & sharing

- Relocation is dynamic, through the segment table
- Shared segments use the same segment number

Allocation

- First-fit / best-fit, as in Part 3
- So: external fragmentation is back



Worked example

Translate three addresses

Seg	Limit	Base
0	1000	1400
1	400	6300
2	400	4300
3	1100	3200
4	1000	4700

<2, 53>

53 < 400 ✓ → 4300 + 53 = **4353**

<3, 852>

852 < 1100 ✓ → 3200 + 852 = **4052**

<0, 1222>

1222 ≥ 1000 ✗ → **trap: addressing error**

Sharing segments

Same editor, different data



Shared segment

Both processes' segment tables point the editor segment at the same place in memory: one copy for everyone.



Private segments

Each process's data segment points somewhere different: its own copy.

sharing is natural with segmentation: just point to the same base

Side by side

Paging vs segmentation

	Paging	Segmentation
Unit size	Fixed (pages / frames)	Variable (segments)
Matches the programmer's view?	No	Yes
Address	(page p, offset d)	(segment s, offset d)
Table entry	Frame number	Base + limit
Fragmentation	Internal	External

Part 06

Segmentation with paging

THINK FIRST

Paging kills external fragmentation; segmentation matches how we think. Why not have both?

discuss for one minute before the next slide

MULTICS

Page the segments

1
(s, d)
from the CPU

→

2
Segment table [s]
holds the base of a page table

→

3
That segment's page table
gives the frame

→

4
Frame + offset
physical address

Problem solved

External fragmentation and lengthy search times for a big enough hole.

The difference

The segment-table entry holds not the segment's base address, but the base address of a page table for that segment.

Pop quiz • click to reveal

Segment 0 has limit 1000 and base 1400. What happens with logical address <0, 1222>?

- **A** Physical address 2622
- **B** Physical address 1222
- **C** Trap: addressing error
- **D** Page fault

Pop quiz • click to reveal

Segment 0 has limit 1000 and base 1400. What happens with logical address <0, 1222>?

A Physical address 2622

B Physical address 1222

C Trap: addressing error

D Page fault

Answer C. 1222 is not less than the limit 1000, so the hardware traps to the OS before adding the base.

Wrap-up

What to remember

OS JOURNAL · CHAPTER 7

- ✓ The CPU uses logical addresses; the MMU turns them into physical ones.
- ✓ Contiguous allocation: first/best/worst fit, and external fragmentation.
- ✓ Segmentation: variable, meaningful segments with base + limit.
- ✓ Swapping parks whole processes on a backing store.
- ✓ Paging: fixed pages in any frames; physical = f × size + d.
- ✓ MULTICS pages each segment to get the best of both.

Next up *Self-study · Virtual Memory and File Systems (see Chapter 1's list)*